CSC3106 Cybersecurity Fundamentals

Mini Project: Data-Driven Security Analysis and Defensive Response

24 July 2026

By Group Q

Assigned Log: 4_auth.log

GitHub Repository:
https://github.com/beansprout09/CSC3106-MiniProject-GroupQ

Name	SIT Student ID	Glasgow Student ID
Haley Tan Hui Xin	2402023	3070674T
Chervelle Tan Eng Tong	2301553	3110761T
Jocasta Tan Li Xuan	2402596	3070698T
Daniel Soong Jia Kang	2403358	3070568S

All members contributed equally

Part 1: Data-Driven Authentication Log Analysis

1. Introduction

1.1 Security Question

This investigation aims to answer the following security question:

> *Which authentication events or patterns within the assigned Linux authentication log should be prioritised for investigation, and what potential risks do they pose to the SSH authentication service and user accounts on the backup01 Linux host?*

1.2 Chosen Asset

The chosen asset is the SSH authentication service (sshd) and associated user accounts on the backup01 Linux host. This asset is important because it controls remote access to the system. If authentication controls are compromised, an unauthorised user may gain access to accounts, modify files, execute commands or disrupt system operations. Although the hostname suggests the system may perform back-up related functions, the organisation has not provided details about stored data or account privileges. Hence, this assessment focuses on the SSH authentication service without assuming the wider business impact of the host.

1.3 Analysis Approach

A Python script was used to analyse the assigned 4_auth.log extract. Regular expressions were used to identify selected OpenSSH authentication records (OpenSSH: Manual Pages, n.d.) and extract the timestamp, event type, username and source IP address from each recognised line.

The analysis included failed password attempts (including invalid usernames), accepted password logins, invalid user events, maximum authentication attempts exceeded, and connections closed during pre-authentication. The extracted data were aggregated to produce summary statistics, identify frequently targeted usernames and source IP addresses, and detect IP addresses exhibiting both failed and successful authentication events. Two Python-generated visualisations were produced: one showing the top source IP addresses by failed authentication attempts, and another illustrating failed authentication activity by hour. All findings and figures were generated directly from the submitted Python script.

2. Python-Verified Findings

The submitted Python script extracted selected OpenSSH authentication events from the supplied 4_auth.log file and generated the evidence used throughout this report. The extracted records included failed password events, accepted password logins, invalid user events, maximum authentication attempts exceeded, and connections closed during pre-authentication. The script also produced summary statistics, tables and visualisations of authentication activity. These outputs were used to identify authentication patterns, assess potential security risks, and support the findings presented in Section 3. A summary of the extracted evidence is provided in Appendix A.

3. Security Interpretation and Risk Prioritisation

3.1 Key Findings

The Python-generated visualisations were analysed to identify notable authentication patterns within the supplied log. Figure 1 illustrates the distribution of failed authentication attempts by source IP, while Figure 2 shows how failed authentication activity varied over the observation period.

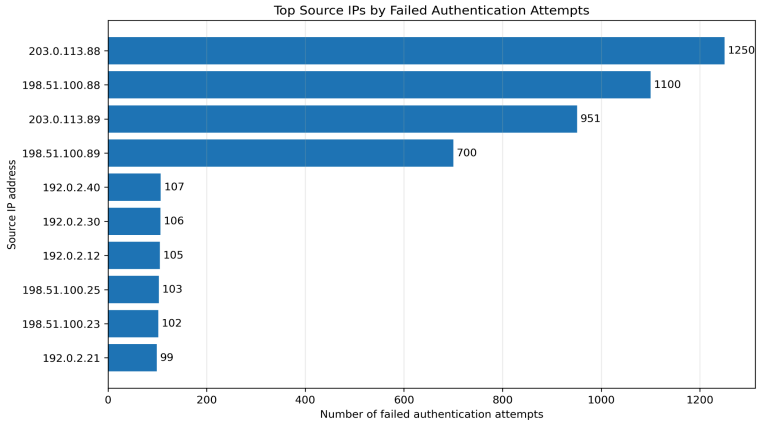

Figure 1. Top source IP addresses by failed authentication events

Figure 1 shows that failed-password activity was highly concentrated among four sources of IP addresses. 203.0.113.88 generated the highest number with 1250 failed-password events, followed by 198.51.100.88 with 1100, 203.0.113.89 with 951 and 198.51.100.89 with 700. Together, these four addresses generated 4001 of the 5176 failed-password events, representing approximately 77.3% of all failed-password activity. In comparison, every other source address generated fewer than 110 failed-password events.

The substantial concentration among these four addresses is consistent with possible automated credential-guessing activity rather than isolated password mistakes by individual users. However, the volume associated with a source IP does not establish who controlled the address or prove that every event was malicious.

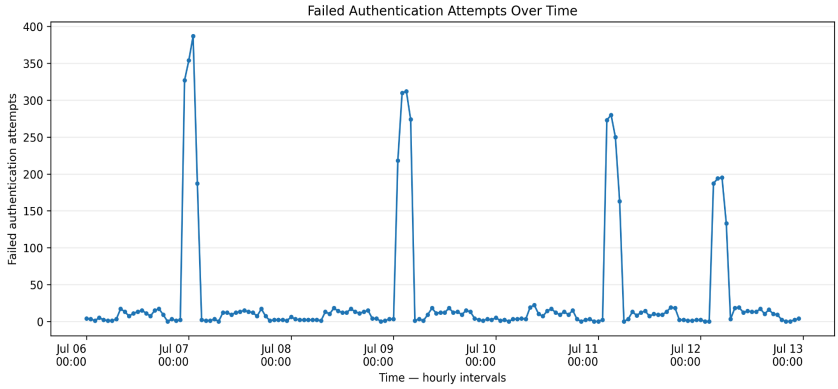

Figure 2. Failed authentication attempts by hourly interval from 6 to 13 July

Figure 2 shows several concentrated bursts of failed password activity around 7, 9, 11, and 12 July. The highest hourly interval contained almost 400 failed attempts, while the volume during

most other periods was considerably lower. The sharp increases and decreases are consistent with possible scripted or automated password guessing. Ordinary user mistakes are less likely to result in hundreds of failed attempts in a short period. Nevertheless, the logs alone cannot confirm the identity or intent of the source.

3.2 Prioritised Risks

The authentication patterns identified in Section 3.1 indicate three primary risks affecting the SSH authentication service and associated user accounts.

The highest-priority risk is password guessing or brute-force attacks against the SSH service. The concentration of failed authentication attempts among a small number of source IP addresses is consistent with repeated credential-guessing activity and presents a high likelihood of continued unauthorised access attempts.

A second risk is the possibility of unauthorised account access following repeated login attempts. One source IP recorded numerous failed authentication attempts before a successful login. Although this pattern warrants further investigation, the available evidence does not confirm that the account was compromised.

The third risk is the operational impact of repeated authentication failures. Large numbers of failed login attempts and repeated authentication-limit events may increase administrative workload, generate excessive alerts, and make genuine attacks more difficult to identify.

A summary of the likelihood, impact and overall risk assessment is provided in Appendix B, while the likelihood-impact matrix is shown in Appendix C.

4. Initial Response Recommendations

The administrator should immediately preserve the relevant logs and validate the accepted backup login from 203.0.113.89 against approved source addresses, VPN records and expected administrative activity. Related SSH sessions, sudo activity, file changes and account modifications should be reviewed. If the login cannot be verified as legitimate, the account should be temporarily restricted, suspicious sessions terminated, and its credentials and SSH keys rotated. Longer-term actions should include disabling direct root login, using key-based authentication and MFA, restricting SSH access to approved networks, and implementing rate limiting or automated blocking.

Part 2: Technical Defensive Response

5. Selected Risk

5.1 Risk from Part 1

Source IP 203.0.113.89 generated 951 failed password attempts and 42 maximum-authentication-attempts events against the SSH authentication service on backup01, before producing one accepted password login to the backup account. The backup account was also the most frequently targeted username in the assigned log, with 1,552 failed authentication attempts. This pattern is consistent with sustained credential-guessing activity followed by a potentially suspicious successful authentication. However, the authentication log alone cannot determine whether the accepted login was authorised or identify the individual responsible for the activity.

5.2 Justification

This finding was selected because it represents the most significant combination of likelihood and potential impact identified in Part 1. The repeated failed authentication attempts indicate persistent credential-guessing behaviour, while the subsequent accepted login from the same source IP warrants immediate investigation. In addition, the activity follows a clear and repetitive pattern, making it well suited to an automated threshold-based detection and response mechanism. The same defensive approach can also be applied to the other high-volume source IPs identified in Part 1.

6. Proposed Technical Defensive Response

6.1 System Overview

To address the risk identified in Part 1, we propose a threshold-based SSH brute-force detection and automated response system. The system monitors SSH authentication events generated by sshd, groups failed login attempts by source IP within a configurable rolling time window, and automatically applies a temporary block when the configured threshold is exceeded.

For this project, the response is implemented as a Python log replay simulation using the authentication data collected in Part 1. This demonstrates how the proposed defence would have responded during the observed attack period without interacting with a live system, complying with the assessment's safe and ethical practice requirements. In a production deployment, the same detection logic could be implemented using tools such as Fail2Ban together with firewall rules (iptables or nftables) to automatically block offending IP addresses (fail2ban, 2024).

6.2 Detection Logic

Failed authentication events are processed chronologically for each source IP. When five failed password attempts occur within a ten-minute rolling window, the source IP is temporarily blocked for 60 minutes. These parameters represent a commonly used baseline for SSH brute-force protection and are defined as configurable constants so they can be adjusted for different deployment environments (fail2ban, 2024).

Any authentication event occurring during the simulated block period is classified as an event that would have been rejected. If an accepted login occurs while the source IP is blocked, it is recorded as a successful authentication that would likely have been prevented under the assumptions of the replay model.

6.3 Implementation

The proposed detection logic was implemented in detect_and_respond.py, which directly imports the Part 1 parse_log() function to ensure both project stages use the same verified dataset. The script processes events chronologically, applies the threshold rule independently for each source IP, and generates three output files: alerts.csv, blocked_events.csv, and detection_summary.csv.

Running the implementation on the assigned log showed that source IP 203.0.113.89 exceeded the detection threshold after five failed login attempts and triggered four temporary blocks during the attack period. The accepted login at 03:08:19 occurred within one of these simulated block periods, indicating that the proposed control would likely have prevented the authentication under the assumptions of the replay model.

Figure 3 shows the failed attempts, the four block windows the control would have triggered, and the accepted login relative to them. The login at 03:08:19 falls inside the second block window, confirming it would have been rejected under this control.

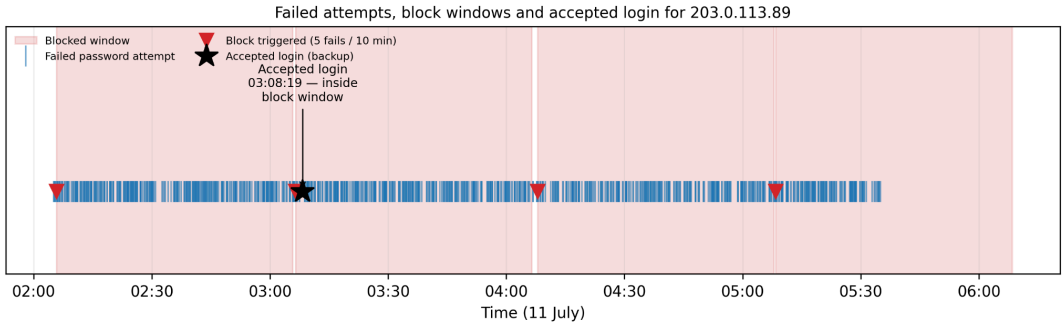

Figure 3. Failed authentication attempts and simulated block windows for source IP 203.0.113.89

7. Evaluation

7.1 Security Benefits

Applying the proposed control with reference to the previously assigned log shows a clear reduction in both the likelihood and the impact of the risk selected in Section 6. Under the proposed threshold, 203.0.113.89 would have been blocked after only 5 failed attempts, rather than being allowed to continue for 951, and would have been blocked four separate times across the attack window as it kept retrying, cutting off the majority of its attempts before they reached the authentication service.

Critically, the accepted login to the backup account at 03:08:19 falls inside the second simulated block window (03:06:31–04:06:31). Under this control, that authentication attempt would have

been completely rejected, removing the exact event that made this finding the highest-priority risk in Part 1.

Beyond blocking, the control improves detection and supports investigation. Each threshold crossing is logged immediately with a timestamp and attempt count (alerts.csv), giving administrators a concrete signal to act on rather than relying on manual review of the full log, while blocked_events.csv and detection_summary.csv provide a per-source-IP record an investigator could use to scope an incident. The same detection logic, applied without modification, also triggered repeated blocks against 203.0.113.88, 198.51.100.88 and 198.51.100.89. The three other addresses responsible for the bulk of failed-password activity identified in Part 1 show that the approach generalises beyond the single finding it was designed around.

7.2 Trade-offs and Limitations

The proposed thresholds (5 attempts within 10 minutes and a 60-minute block) follow a widely used baseline, such as Fail2Ban's default sshd jail, but were not tuned against a labelled baseline of this environment's normal login behaviour. A legitimate user who mistypes their password five times within ten minutes could also be temporarily blocked, creating a false-positive risk that the organisation would need to manage by adjusting the detection thresholds to suit its operational environment (OWASP Foundation, n.d.).

The control is also IP-based, so an attacker who rotates source addresses, for example using a botnet or proxy pool, would not be stopped by it; it removes the specific low-effort, single-source brute-force path observed in this log rather than credential-guessing in general. It similarly does not address the underlying credential weakness, as the backup account remains reachable with only a password. Therefore, it buys time and visibility rather than replacing the Part 1 recommendation to rotate the account's credentials or SSH keys and move to key-based authentication or MFA (OWASP Foundation, n.d.).

Finally, the control is limited to the authentication layer and to a log-replay simulation. The assigned log extract contains no post-authentication activity, so it cannot confirm if anything occurred during the one login that was actually accepted before the simulated block would have applied to later attempts; closing that gap requires investigating backup01 directly, as recommended in Part 1 (Initial Response Recommendations). Likewise, the result demonstrates what an automated blocking mechanism would have done against historical evidence, in line with the assessment's requirement not to interact with a live system; deploying the same logic in production (e.g. via Fail2Ban with iptables/nftables) and validating it against live traffic remains a follow-on step outside this assessment's scope.

References

OpenSSH. (n.d.). *OpenSSH manual pages*. https://www.openssh.org/manual.html

fail2ban. (2024, April 25). *fail2ban: Daemon to ban hosts that cause multiple authentication errors* [Computer software]. GitHub. https://github.com/fail2ban/fail2ban

OWASP Foundation. (n.d.). *Authentication Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Appendix A: Summary of Authentication Log Evidence

Observation	Evidence
Failed login attempts	5,176 failed password events
Most active source IP	203.0.113.88 (1250 failed password events)
Most targeted account	backup (1552 failed attempts)
Maximum authentication attempts	42 “maximum authentication attempts exceeded” events
Failed then successful login	203.0.113.89 recorded both failed and accepted authentication events

Appendix B: Risk Matrix for SSH Authentication Service

Prioritised Risk	Evidence from Analysis	Likelihood	Impact	Risk Rating
Password guessing / brute-force attacks against SSH	5176 failed login attempts observed across multiple source IPs. The highest source IP generated 1250 failed attempts.	High	High	High
Possible account compromise following repeated login attempts	Source IP 203.0.113.89 recorded 951 failed login attempts followed by one successful login to the backup account. This pattern warrants further investigation but does not confirm unauthorised access.	Medium	High	High
Repeated authentication failures affecting monitoring and administration	42 “maximum authentication attempts exceeded” events and numerous failed logins increase investigation workload and may hide genuine attacks.	Medium	Medium	Medium

Appendix C: Likelihood x Impact Matrix

	Low Impact	Medium Impact	High Impact
High Likelihood			Password guessing / brute-force attacks
Medium Likelihood		Authentication monitoring burden	Possible account compromise
Low Likelihood			

Appendix D: AI Use Declaration

AI tools used: ChatGPT (OpenAI) and Claude (Claude Code, Anthropic).

Used for:

Part 1 (ChatGPT): Used to improve the clarity and readability of the report, suggest wording, answer questions about referencing, and provide feedback on the report structure. The group designed the analysis approach, wrote and ran analysis.py, and produced all findings, figures, risk assessments, and recommendations directly from its outputs.

Part 2 (Claude Code): Used in a collaborative process to debug a timestamp display issue in the output CSV files and assist with the implementation of plot_focus_ip_timeline() (Figure 3). The group specified the required behaviour and reviewed and revised the implementation throughout the process. Claude Code also drafted initial wording for parts of the evaluation section, which the group edited before inclusion.

How outputs were checked:

All quantitative findings, figures, tables, and statistics were verified by re-running the submitted scripts (analysis.py and detect_and_respond.py) against the assigned 4_auth.log file and checking the generated outputs directly. No AI-suggested figure, numerical result, or claim was included without this verification.

Parts substantially shaped by AI:

For Part 1, AI assisted with improving the wording, readability, and organisation of the written report but did not generate the analysis, findings, figures, or conclusions. For Part 2, AI assisted with debugging a timestamp display issue, provided assistance with the implementation of the plot_focus_ip_timeline() function used to generate Figure 3, and suggested initial wording for parts of the evaluation section. All analysis, technical decisions, implementation, interpretation of results, and final report content were completed, verified, and approved by the group.

Ensuring Python-verified evidence:

No figures, counts, or claims generated or suggested by AI were included in the report without first being reproduced by re-running the submitted script against the assigned 4_auth.log and confirming the output files matched.